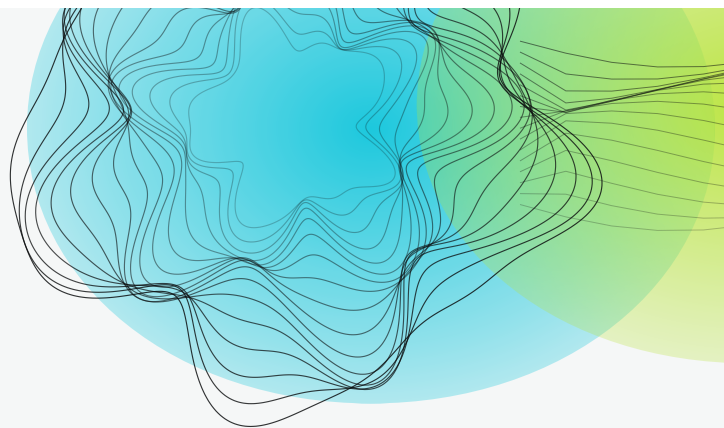




MASTÈRE CYBERSÉCURITÉ — 4^e ANNÉE · GESTION DES CONFIGURATIONS & IAC

MODULE 2

Gestion des configurations et Git

Du CMDB d'ITIL au dépôt Git : ce que « gérer une configuration » veut dire, comment Git le rend possible, et pourquoi un dépôt mal tenu est la première porte d'entrée d'un attaquant dans une infrastructure.

JOUR 1 — 3 H 30

NIVEAU : SOCLE

PRATIQUE : 1 H 30

Formateur : Boris Rose

Gestion des configurations et Infrastructure as Code
Terraform · Ansible · Docker · CI/CD



Sommaire

Ce que « gestion de configuration » veut dire

- Un même terme, deux métiers
- Le vocabulaire de référence
- Provisionner, configurer, orchestrer
- Pourquoi c'est un sujet de cybersécurité

Git : le modèle mental

- Ce que Git stocke réellement
- Les quatre zones
- Les commandes du quotidien
- Branches et tags : la différence qui compte pour la sécurité

Structurer un dépôt IaC

- Arborescence de référence
- .gitignore : le premier rempart
- Messages de commit : la convention Conventional Commits
- Protection des branches

Les secrets dans Git : le problème central

- Pourquoi git rm ne suffit pas
- Détecter avant de commiter
- Outils de détection de secrets
- Purger un secret de l'historique

Signer ses commits

- Le problème : l'identité Git n'est pas authentifiée
- Signature GPG ou SSH

TP 1 — Un dépôt IaC sain, de bout en bout

CHAPITRE 1

Ce que « gestion de configuration » veut dire

◎ OBJECTIFS DU CHAPITRE

À l'issue de ce chapitre, vous saurez :

- distinguer les **deux acceptions** du terme « gestion de configuration », qui coexistent dans le métier et désignent des choses différentes ;
- définir précisément **élément de configuration**, **CMDB**, **référentiel (baseline)** et **dérive de configuration** ;
- expliquer pourquoi la gestion de configuration est un **prérequis de la sécurité**, et non un sujet purement organisationnel ;
- situer Terraform, Ansible et Git les uns par rapport aux autres.

Un même terme, deux métiers

Le mot piège dès la première heure. Selon l'interlocuteur, « gestion de configuration » désigne :

◆ DÉFINITION — GESTION DE CONFIGURATION — SENS 1 : GOUVERNANCE (ITIL, ITSM)

Processus **organisationnel** consistant à identifier, enregistrer, suivre et contrôler tous les composants d'un système d'information et leurs relations. Son artefact central est la **CMDB**. On y répond à des questions comme : « quelles applications tombent si ce serveur s'arrête ? », « qui a modifié ce pare-feu, quand, sur quelle demande ? ».

C'est le sens qu'emploient un auditeur, un RSSI ou un responsable des changements.

◆ DÉFINITION — GESTION DE CONFIGURATION — SENS 2 : OUTILLAGE (DEVOPS)

Pratique **technique** consistant à décrire l'état souhaité des systèmes dans du code, puis à laisser un outil (**Ansible**, Puppet, Chef, SaltStack) amener et maintenir les machines dans cet état.

C'est le sens qu'emploie un ingénieur d'exploitation quand il dit « on gère la configuration avec Ansible ».

★ LES DEUX SENS SE REJOIGNENT EN IAC

Quand l'infrastructure est décrite dans un dépôt Git, **le dépôt devient la CMDB** — mais une CMDB exécutable, qui ne peut pas mentir puisqu'elle est ce qui construit réellement le système. C'est la promesse la plus forte de l'IaC, et c'est aussi ce qui rend le dépôt critique : il concentre désormais la connaissance **et** le pouvoir sur l'infrastructure.

Le vocabulaire de référence

◆ DÉFINITION — ÉLÉMENT DE CONFIGURATION (CI — CONFIGURATION ITEM)

Tout composant du SI devant être géré pour délivrer un service : un serveur, une machine virtuelle, une adresse IP, un certificat TLS, une règle de pare-feu, une version d'application, un contrat de support.

Attention à l'homonymie : dans ce cours, **CI** signifiera aussi **Continuous Integration** (intégration continue, module 6). Le contexte tranche, mais le faux ami est réel.

◆ DÉFINITION — CMDB (CONFIGURATION MANAGEMENT DATABASE)

Base de données recensant les éléments de configuration et surtout **leurs relations**. Sa valeur ne vient pas de la liste mais du graphe : c'est lui qui permet l'analyse d'impact.

Le problème historique des CMDB est qu'elles sont alimentées **manuellement** et divergent donc de la réalité. Une CMDB fausse est pire qu'une absence de CMDB, parce qu'on lui fait confiance.

◆ DÉFINITION — RÉFÉRENTIEL / LIGNE DE BASE (BASELINE)

État de référence, **approuvé et figé**, d'un ensemble d'éléments de configuration à un instant donné. C'est la référence par rapport à laquelle on mesure tout écart, et l'état vers lequel on revient en cas de problème.

En IaC, la baseline est un **commit Git** — identifié par une empreinte cryptographique, donc infalsifiable et reproductible.

◆ DÉFINITION — DÉRIVE DE CONFIGURATION (CONFIGURATION DRIFT)

Écart accumulé entre l'état **décrit** d'un système et son état **réel**, causé par des modifications hors processus : correctif appliqué à chaud, règle de pare-feu ouverte « juste pour tester », paquet installé manuellement pendant un incident.

La dérive est **inévitable** : elle ne se combat pas par la discipline mais par la **détection automatique** (`terraform plan`, mode `--check` d'Ansible) et par l'**immutabilité** (on remplace au lieu de modifier).

🕒 LE SERVEUR QU'ON N'OSE PLUS TOUCHER

Chaque organisation a le sien : une machine en production depuis des années, configurée à la main par quelqu'un qui est parti, que personne ne sait reconstruire. On applique les correctifs en croisant les doigts, on ne la redémarre jamais.

La communauté a nommé ce phénomène le **snowflake server** — un serveur unique, non reproductible, comme un flocon de neige. Le texte de popularisation le plus cité est celui de **Chad Fowler**, « **Trash Your Servers and Burn Your Code** » (23 juin 2013) :

« If you absolutely know a system has been created via automation and never changed since the moment of creation, most of the problems I describe above disappear. »

Fowler ne revendique d'ailleurs pas la paternité de l'idée (« *We're obviously not the first ones to think of it* ») — c'est un texte de popularisation, pas un acte de naissance.

Conséquence en sécurité : un serveur qu'on n'ose pas redémarrer est un serveur qu'on ne corrige pas. La dérive de configuration produit mécaniquement de la dette de sécurité.

Provisionner, configurer, orchestrer

Trois verbes souvent confondus, et qui correspondent à trois familles d'outils :

COUCHE	QUESTION	OUTILS	VU AU MODULE
Provisionnement	Quelles ressources doivent exister ? (VM, réseau, base, rôle IAM)	Terraform , OpenTofu, CloudFormation, Bicep, Pulumi	3
Configuration	Que doit contenir chaque machine ? (paquets, fichiers, services, utilisateurs)	Ansible , Puppet, Chef, SaltStack	5
Orchestration	Comment les charges tournent-elles et se coordonnent-elles ?	Kubernetes, ECS, Nomad, Container Apps	4

★ LA FRONTIÈRE EST UNE DÉCISION D'ARCHITECTURE, PAS UNE RÈGLE

Terraform peut lancer un script au démarrage d'une VM (*user data*) et donc « configurer ». Ansible dispose de modules `amazon.aws.ec2_instance` et peut donc « provisionner ».

La ligne de partage que ce cours retient, et qui est la plus courante en production :

Terraform crée ce qui a un cycle de vie propre au fournisseur cloud. Ansible façonne l'intérieur des machines.

Nous verrons au module 5 comment les articuler proprement.

Pourquoi c'est un sujet de cybersécurité

★ LES QUATRE APPORTS DIRECTS À LA SÉCURITÉ

- 1. Traçabilité.** Chaque changement d'infrastructure porte un auteur, une date, un motif et une revue. C'est la base de tout audit, et c'est le principe d'**enregistrement de la compromission** (*compromise recording*) évoqué par Saltzer & Schroeder.
- 2. Reproductibilité.** Après un incident, on reconstruit à l'identique depuis un commit connu et sain, au lieu de restaurer une sauvegarde dont on ignore si elle contient déjà la compromission.
- 3. Contrôle *a priori*.** Le code peut être analysé **avant** d'être appliqué (`terraform plan` , scan Checkov/Trivy, politique OPA). C'est le seul moment où une erreur ne coûte rien.
- 4. Réduction de la surface humaine.** Moins d'accès interactifs en production, donc moins de comptes à privilèges permanents — le vecteur d'attaque n° 1 selon les rapports d'incident.

◆ UN CHIFFRE À GARDER EN TÊTE

Le **Global Incident Response Report 2026 d'Unit 42** (750 incidents majeurs investigués, 50 pays) indique que dans **plus de 90 % des incidents**, des **mauvaises configurations ou des lacunes de couverture** ont matériellement permis l'attaque. Les faiblesses d'identité apparaissent dans environ **90 %** des investigations.

Autrement dit : la grande majorité des compromissions n'exploite pas une vulnérabilité logicielle inconnue, mais une **configuration**. Ce cours porte donc exactement sur la surface d'attaque dominante.

Source : <https://www.paloaltonetworks.com/blog/2026/02/unit-42-global-ir-report/>

CHAPITRE 2

Git : le modèle mental

◎ OBJECTIFS DU CHAPITRE

- Comprendre que Git stocke des **instantanés**, pas des différences.
- Distinguer les quatre zones : copie de travail, index, dépôt local, dépôt distant.
- Savoir ce qu'est un **commit**, une **branche**, un **tag**, et pourquoi un tag est **mutable**.

Ce que Git stocke réellement

◆ DÉFINITION — GIT

Système de gestion de versions **distribué**, créé par Linus Torvalds en 2005 pour le noyau Linux. « Distribué » signifie que chaque copie du dépôt contient l'**historique complet** : il n'y a pas de serveur central indispensable au fonctionnement.

★ GIT NE STOCKE PAS DES DIFFÉRENCES

Contrairement à SVN ou CVS, Git enregistre à chaque commit un **instantané complet** de l'arborescence. Les fichiers inchangés ne sont pas dupliqués : le commit pointe simplement vers le même objet qu'avant.

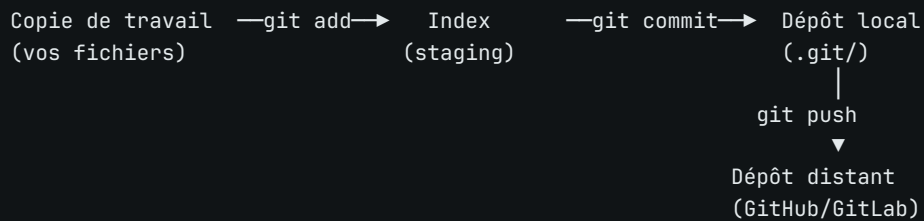
Chaque objet est identifié par l'**empreinte cryptographique de son contenu** (SHA-1, avec une transition en cours vers SHA-256). Deux conséquences majeures :

1. **L'historique est infalsifiable** : modifier un commit ancien change son empreinte, donc celle de tous ses descendants. La falsification est immédiatement visible.
2. **Un secret commité l'est pour toujours** : il reste dans l'objet, atteignable par son empreinte, même après suppression du fichier dans un commit ultérieur. Ceci est développé au chapitre 5.

Les quatre types d'objets :

OBJET	CONTENU
blob	le contenu binaire d'un fichier (sans son nom)
tree	un répertoire : liste de noms → blobs et trees
commit	un tree racine + parent(s) + auteur + date + message
tag annoté	un pointeur nommé vers un objet, avec message et signature possible

Les quatre zones



◆ DÉFINITION — INDEX (OU STAGING AREA)

Zone intermédiaire où l'on **compose** le prochain commit. C'est la particularité de Git : on ne commit pas « ce qu'on a modifié », on commit « ce qu'on a explicitement ajouté à l'index ».

Cela permet de découper un travail en plusieurs commits cohérents (`git add -p` pour choisir morceau par morceau), au lieu de produire un commit fourre-tout impossible à relire — et impossible à auditer.

Les commandes du quotidien

```

git init                # créer un dépôt
git clone <url>         # copier un dépôt distant

git status              # état - à taper sans arrêt
git diff                # copie de travail vs index
git diff --staged       # index vs dernier commit

git add fichier.tf      # ajouter à l'index
git add -p              # ajouter interactivement, morceau par morceau
git commit -m "feat: ajoute le VPC de préproduction"

git log --oneline --graph --decorate --all
git show <sha>          # contenu d'un commit

git switch -c feature/vpc # créer et basculer sur une branche
git switch main          # revenir
git merge feature/vpc    # fusionner
git rebase main           # rejouer ses commits sur main

git fetch               # récupérer sans modifier son travail
git pull                # fetch + merge (ou rebase selon config)
git push -u origin feature/vpc
  
```

▲ `GIT PULL` N'EST PAS ANODIN

`git pull` = `git fetch` + `git merge`. Sur une branche partagée, cela crée des commits de fusion parasites et peut déclencher des conflits au pire moment.

Prenez l'habitude de :


```
git fetch                # je regarde ce qui est arrivé
git log --oneline HEAD..origin/main # ce que je n'ai pas encore
git rebase origin/main   # je rejoue mon travail par-dessus
```

Et configurez une bonne fois :

```
git config --global pull.rebase true
```

Branches et tags : la différence qui compte pour la sécurité

◆ DÉFINITION — BRANCHE

Un **pointeur mobile** vers un commit. Créer une branche coûte 41 octets : c'est un fichier contenant une empreinte. C'est pourquoi Git encourage à en créer beaucoup.

◆ DÉFINITION — TAG

Un pointeur **nommé** vers un commit, destiné à marquer une version (`v1.4.2`).

Un **tag est techniquement mutable** : rien n'empêche `git tag -f v1.0 <autre-commit>` puis `git push --force --tags`. Le nom est le même, le contenu pointé est différent. Ce n'est pas un bug de Git : c'est une propriété du modèle, et c'est une des surfaces d'attaque les plus exploitées de l'écosystème DevOps.

◆ TJ-ACTIONS/CHANGED-FILES — 14 MARS 2025 (CVE-2025-30066)

Le cas d'école de la mutabilité des tags.

`tj-actions/changed-files` est une GitHub Action très populaire (utilisée par plus de 23 000 dépôts). Le 14 mars 2025, vers 9 h heure du Pacifique, un attaquant ayant compromis le jeton d'accès personnel du bot du projet a **réécrit l'intégralité des tags de version — de `v1.0.0` à `v44 / v45` — pour qu'ils pointent tous vers un unique commit malveillant** (`0e58ed86...`).

La charge : un script téléchargeait `memdump.py` depuis un gist. Ce script parcourait `/proc`, localisait le processus `Runner.Worker`, lisait `/proc/[pid]/maps` puis `/proc/[pid]/mem`, et y cherchait le motif interne par lequel le runner GitHub représente ses secrets :

```
"[^"]+":{"value":"[^"]*", "isSecret":true}
```

Il encodait le résultat **deux fois en base64** — pour échapper au *secret scanning* de GitHub, qui ne décode pas récursivement — puis l'affichait dans les **logs du workflow**.

Ce n'était donc pas une lecture de variables d'environnement mais un **dump de la mémoire du runner** : d'où la capture de secrets qui n'étaient même pas exposés à l'étape compromise. Clés AWS, jetons GitHub, jetons npm, clés privées RSA.

Le risque différencié (formulation de Wiz) : « *The primary risk is to public repositories, where secrets dumped into workflow logs are public as well; conversely, the risk to private repos is limited.* »

Précision importante : le chiffre de « 23 000 dépôts » souvent cité est le nombre de dépôts **utilisant** l'action, **pas** le nombre ayant fuité des secrets. La fenêtre a duré environ 24 h ; Wiz ne recense que « des dizaines » de dépôts publics réellement touchés.

Ce qui protégeait totalement : épingler l'action par **empreinte de commit complète** au lieu du tag.

```
- uses: tj-actions/changed-files@v45 # ✗ tag mutable
- uses: tj-actions/changed-files@a1b2c3d4e5f6... # ✓ commit immuable
```

Sources : StepSecurity (découverte initiale) <https://www.stepsecurity.io/blog/harden-runner-detection-tj-actions-changed-files-action-is-compromised> · Wiz <https://www.wiz.io/blog/github-action-tj-actions-changed-files-supply-chain-attack-cve-2025-30066> · alerte CISA du 18 mars 2025 · NVD CVE-2025-30066

★ LE PRINCIPE GÉNÉRALISABLE

Un nom lisible est un pointeur mutable. Une empreinte de contenu est une identité.

Ce principe se décline partout dans ce cours :

DOMAINE	MUTABLE (À ÉVITER)	IMMUABLE (À PRÉFÉRER)
Git	<code>main</code> , <code>v45</code>	empreinte de commit
GitHub Actions	<code>uses: org/action@v4</code>	<code>uses: org/action@<sha40></code>
Docker	<code>nginx:latest</code>	<code>nginx@sha256:...</code>
Terraform	<code>version = ">= 5.0"</code>	<code>.terraform.lock.hcl</code>
npm / PyPI	<code>^1.2.0</code>	fichier de verrouillage

CHAPITRE 3

Structurer un dépôt IaC

◎ OBJECTIFS DU CHAPITRE

Mettre en place les fichiers de gouvernance sans lesquels un dépôt d'infrastructure devient rapidement inexploitable — ou dangereux.

Arborescence de référence

```
infra-projet/
├── .github/
│   └── workflows/
│       └── terraform.yml          # pipeline CI/CD (module 6)
├── .gitattributes                # fins de ligne (module 1)
├── .gitignore                   # ce qui ne doit JAMAIS être versionné
├── .pre-commit-config.yaml      # garde-fous locaux
├── .editorconfig
├── Makefile                     # interface unique du projet
├── README.md                   # comment démarrer, en 10 lignes
├── docs/
│   └── adr/                     # décisions d'architecture datées
├── modules/                     # modules Terraform réutilisables
│   ├── network/
│   └── compute/
├── envs/                        # une racine Terraform par environnement
│   ├── dev/
│   ├── staging/
│   └── prod/
├── ansible/
│   ├── inventories/
│   ├── roles/
│   └── playbooks/
├── docker/
│   └── web/Dockerfile
```

`.gitignore` : le premier rempart

▲ LE `.GITIGNORE` DOIT EXISTER AVANT LE PREMIER `TERRAFORM INIT`

`terraform init` crée un répertoire `.terraform/` de plusieurs centaines de mégaoctets, et `terraform apply` produit un `terraform.tfstate` contenant des secrets en clair (module 3). Si le `.gitignore` arrive après, ils sont déjà dans l'historique.

GitHub publie un `.gitignore` officiel pour Terraform, à compléter :

```
# ---- Terraform -----
**/.terraform/*
*.tfstate
```

```

*.tfstate.*
crash.log
crash.*.log
*.tfvars
*.tfvars.json
override.tf
override.tf.json
*_override.tf
*_override.tf.json
.terraformrc
terraform.rc
.terraform.tfstate.lock.info

# On VERSIONNE en revanche le fichier de verrouillage des providers :
!.terraform.lock.hcl

# ---- Secrets et environnements -----
.env
.env.*
*.pem
*.key
*.p12
*.pfx
credentials
.aws/
.azure/
*.retry

# ---- Systèmes -----
.DS_Store
Thumbs.db
.vscode/
.idea/

```

★ `*.TFVARS` IGNORÉ, MAIS `.TERRAFORM.LOCK.HCL` VERSIONNÉ

Deux décisions à comprendre :

- `*.tfvars` est ignoré car ces fichiers contiennent régulièrement des valeurs sensibles. Corollaire : versionnez un `terraform.tfvars.example` documenté et sans valeur réelle, pour que le projet reste démarrable.
- `.terraform.lock.hcl` est versionné car il fige les versions et les empreintes des providers. C'est l'équivalent d'un `package-lock.json`. Ne pas le versionner, c'est accepter qu'un `terraform init` chez un collègue installe une version différente.

Messages de commit : la convention *Conventional Commits*

```

<type>(<portée>): <description à l'impératif, en minuscules>

<corps optionnel : le POURQUOI, pas le QUOI>

<pied optionnel : BREAKING CHANGE, refs #123>

```

TYPE	USAGE
<code>feat</code>	nouvelle capacité d'infrastructure
<code>fix</code>	correction
<code>refactor</code>	réorganisation sans changement de comportement
<code>chore</code>	maintenance (dépendances, outillage)
<code>docs</code>	documentation
<code>ci</code>	pipeline
<code>security</code>	correctif ou durcissement de sécurité

```
feat(network): ajoute un sous-réseau privé en eu-west-3c
```

Le sous-réseau public actuel expose les instances applicatives.
On introduit un sous-réseau privé avec NAT gateway pour les sorties.

Refs: TICKET-482

★ LE MESSAGE DE COMMIT EST UNE PIÈCE D'AUDIT

Six mois après, lors d'un audit ou d'un post-mortem, personne ne relira le diff. On lira le message. « fix » ou « update » ne dit rien ; « corrige la règle de sécurité qui exposait 0.0.0.0/0 sur le port 22 » raconte l'histoire.

Protection des branches

Sur la branche par défaut d'un dépôt d'infrastructure, exigez au minimum :

- interdiction du *push* direct (tout passe par *pull request*) ;
- au moins une revue approuvée par une personne différente de l'auteur ;
- vérifications de CI obligatoires (format, validation, scan de sécurité, plan) ;
- interdiction du *force push* et de la suppression ;
- si possible, **commits signés obligatoires** (chapitre suivant).

◆ ASYNCAPI — 14 JUILLET 2026 : PROTÉGER `MAIN` NE SUFFIT PAS

Un attaquant a obtenu un accès en écriture sur la branche `next` du dépôt AsyncAPI. Il n'a pas eu besoin de voler un jeton npm : il a laissé **le pipeline légitime du projet publier pour lui**. StepSecurity le formule ainsi :

« the attacker didn't steal an npm token: they gained push access to the repository's next branch and let the project's real CI/CD pipeline do the publishing for them »

Les paquets malveillants portaient donc des **attestations de provenance SLSA parfaitement valides**.

Le vecteur initial : parmi **37 pull requests** ouvertes (la plupart en couverture), **une seule** exploitait un workflow déclenché par `pull_request_target` qui faisait un checkout du code de la PR — permettant de voler un jeton hautement privilégié. Détail accablant : **un contributeur avait proposé le correctif le 17 mai ; il était encore non fusionné 58 jours plus tard.**

Les trois leçons : 1. La protection de branche doit couvrir **toutes** les branches qui déclenchent une publication, pas seulement `main`. 2. Une attestation de provenance prouve **d'où** vient l'artefact, pas qu'il est sain. 3. Un correctif de sécurité non fusionné est une vulnérabilité ouverte. Fixez un délai de traitement des PR de sécurité.

Sources : <https://www.stepsecurity.io/blog/compromised-next-branch-pushes-malicious-asyncapi-generator-generator-helpers-and-generator-components-to-npm> · <https://www.wiz.io/blog/m-red-team-asyncapi-supply-chain-compromise-via-github-actions>

🔗 EXERCICE 2.1 — INITIALISER UN DÉPÔT IAC DANS LES RÈGLES

Créez un dépôt local `tp-iac` contenant :

1. `.gitattributes` normalisant les fins de ligne (module 1) ;
2. `.gitignore` couvrant Terraform et les secrets ;
3. `README.md` avec un titre, une section « Prérequis » et une section « Démarrage » ;
4. `Makefile` avec une cible `help` par défaut ;
5. trois commits **séparés**, respectant *Conventional Commits*.

Vérifiez avec `git log --oneline --graph`, puis avec `git show --stat HEAD` que chaque commit ne contient que ce qu'il annonce.

CHAPITRE 4

Les secrets dans Git : le problème central

◎ OBJECTIFS DU CHAPITRE

- Comprendre pourquoi supprimer un secret ne le supprime pas.
- Détecter les secrets **avant** qu'ils n'entrent dans l'historique.
- Savoir réagir correctement à une fuite.

Pourquoi `git rm` ne suffit pas

Reprenons le modèle objet : un commit référence un *tree*, qui référence des *blobs*. Le blob contenant le secret **existe toujours** dans `.git/objects`, identifié par son empreinte. Il est atteignable par :

```
git log --all --full-history -- chemin/du/fichier
git show <sha-ancien>:chemin/du/fichier
```

Et si le dépôt a été poussé, il existe sur le serveur **et sur toutes les copies clonées**. Sur GitHub, il reste accessible via les URL de commit même après suppression de la branche.

★ LA SEULE BONNE RÉACTION À UN SECRET POUSSÉ

1. **Révoquer immédiatement** le secret chez l'émetteur (console AWS, Entra ID, registre...). C'est l'action n° 1 : elle prend trente secondes et neutralise la fuite.
2. **Générer** un nouveau secret et le stocker dans un coffre.
3. **Chercher les traces d'usage** dans les journaux (CloudTrail, journaux d'audit Entra ID) — le secret a peut-être déjà servi.
4. **Puis, éventuellement**, réécrire l'historique.

Réécrire l'historique **sans révoquer d'abord** est une erreur classique : cela donne un faux sentiment de sécurité alors que le secret circule déjà. Les moissonneurs automatiques scannent les dépôts publics en continu — le délai entre publication et exploitation se compte en minutes.

◆ EMERALDWHALE — SYSDIG, 30 OCTOBRE 2024

Une campagne a scanné massivement Internet à la recherche d'un seul chemin : `/.git/config` exposé publiquement par des serveurs web mal configurés. L'outil utilisé était `httpx`, un scanner légitime.

Le fichier `.git/config` contient souvent l'URL du dépôt distant, parfois **avec les identifiants intégrés** (`https://user:token@github.com/...`). À partir de là, les opérateurs clonaient les dépôts privés et en extrayaient par expressions régulières les clés `AKIA...`.

Le butin : plus de **15 000 identifiants cloud**, environ 2 000 jetons GitHub valides sur 6 000 collectés, des identifiants issus de plus de **10 000 dépôts privés**.

Le détail savoureux et pédagogiquement précieux : les données volées étaient stockées dans un bucket S3 (`s3simplisitter`, plus d'un téraoctet) **lui-même exposé publiquement**. L'infrastructure de l'attaquant reproduisait exactement la faute qu'il exploitait.

Le contrôle : ne jamais servir le répertoire `.git/` depuis un serveur web. En nginx :

```
location ~ /\.git { deny all; return 404; }
```

Source : <https://sysdig.com/blog/emeraldwhale/>

◆ EXTORSION VIA FICHIERS `.ENV` — UNIT 42, 15 AOÛT 2024

Campagne à très grande échelle : **plus de 230 millions de cibles uniques scannées, 110 000 domaines** touchés, plus de 90 000 variables d'environnement exfiltrées dont 7 000 identifiants de services cloud.

La chaîne : fichier `.env` exposé → clés AWS → création de rôles IAM `AdministratorAccess` → déploiement de fonctions Lambda malveillantes multi-régions → extorsion par **suppression des objets S3**.

Source : <https://unit42.paloaltonetworks.com/large-scale-cloud-extortion-operation/>

Détecter avant de commiter

◆ DÉFINITION — PRE-COMMIT

Framework de gestion de *hooks* Git. Un fichier `.pre-commit-config.yaml` versionné décrit les vérifications à exécuter ; chaque développeur installe les hooks une fois avec `pre-commit install`. Les mêmes vérifications sont ensuite rejouées en CI/CD — garantissant que le contrôle local et le contrôle serveur sont identiques.

```
pipx install pre-commit      # ou : pip install --user pre-commit
cd mon-depot
pre-commit install           # installe le hook dans .git/hooks/
pre-commit run --all-files   # première passe sur tout le dépôt
```

.PRE-COMMIT-CONFIG.YAML — CONFIGURATION DE RÉFÉRENCE POUR UN PROJET IAC

```
repos:
  # --- Hygiène générale -----
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v5.0.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-json
      - id: check-merge-conflict
      - id: mixed-line-ending
```



```

    args: ["--fix=lf"]
  - id: check-added-large-files
    args: ["--maxkb=500"]
  - id: detect-private-key      # clés privées PEM

# ---- Détection de secrets -----
- repo: https://github.com/gitleaks/gitleaks
  rev: v8.24.0
  hooks:
    - id: gitleaks

# ---- Terraform -----
- repo: https://github.com/antonbabenko/pre-commit-terraform
  rev: v1.99.0
  hooks:
    - id: terraform_fmt
    - id: terraform_validate
    - id: terraform_docs
    - id: terraform_tflint
    - id: terraform_trivy

# ---- Ansible -----
- repo: https://github.com/ansible/ansible-lint
  rev: v25.6.1
  hooks:
    - id: ansible-lint

```

▲ LES HOOKS LOCAUX NE SONT PAS UN CONTRÔLE DE SÉCURITÉ

Un hook `pre-commit` s'exécute **sur le poste du développeur**, et se contourne avec `git commit --no-verify`. C'est un **filet de sécurité ergonomique**, pas une barrière.

La barrière doit être **côté serveur** : les mêmes vérifications rejouées dans le pipeline de CI/CD, avec une protection de branche qui rend leur succès obligatoire pour fusionner.

C'est, une fois encore, le principe de **médiation complète** : un contrôle qui peut être contourné par un autre chemin ne protège pas.

Outils de détection de secrets

OUTIL	ÉDITEUR	PARTICULARITÉ
gitleaks	communauté	Rapide, règles personnalisables, scanne l'historique complet, intégration pre-commit et CI
TruffleHog	Truffle Security	Vérifie les secrets trouvés en les testant contre les API — beaucoup moins de faux positifs
detect-secrets	Yelp	Fonctionne avec une <i>baseline</i> audité : on marque les faux positifs une fois pour toutes
git-secrets	AWS Labs	Historique, orienté clés AWS ; peu actif

```

gitleaks detect --source . --verbose      # scanne tout l'historique
gitleaks protect --staged                  # scanne seulement l'index

```

◆ TRUFFLEHOG RETOURNÉ CONTRE SES UTILISATEURS — VER « SHAI-HULUD », NPM, SEPTEMBRE 2025

Un point remarquable qui doit vous marquer : le ver npm **Shai-Hulud** (première vague à partir du 15 septembre 2025) **téléchargeait et exécutait TruffleHog lui-même**.

Le code malveillant, injecté via un hook `postinstall`, lançait :

```
trufflehog filesystem / --json
```

— c'est-à-dire un scan de **l'intégralité du système de fichiers** de la victime, à la recherche de `GITHUB_TOKEN`, `NPM_TOKEN`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`. Il interrogeait aussi les API de métadonnées d'instance (IPv4 `169.254.169.254` et IPv6 `[fd00:ec2::254]` pour AWS, `metadata.google.internal` pour GCP).

L'exfiltration passait par un **dépôt GitHub public** créé chez la victime, nommé `Shai-Hulud` (Socket : « *public by default to ensure easy access for the command and control infrastructure* »), par un webhook, et par un workflow malveillant `.github/workflows/shai-hulud-workflow.yml`. Les données étaient encodées **en double base64** pour échapper au secret scanning de GitHub.

Le ver se propageait ensuite en interrogeant l'API npm pour lister les paquets du mainteneur compromis et en publier des versions trojanisées. Volumétrie de la première vague : de 40 paquets le 15 septembre à **plus de 500** trois jours plus tard.

Une **seconde vague**, « **Shai-Hulud: The Second Coming** » (avec le chiffre `1`), a frappé fin novembre 2025 : environ **700 paquets npm**, plus de **25 000 dépôts GitHub malveillants**, et l'exfiltration confirmée de **775 jetons GitHub**, **373 identifiants AWS**, **300 GCP** et **115 Azure**. Elle utilisait `preinstall` au lieu de `postinstall` — pour s'exécuter même si l'installation échoue ensuite — et **détruisait le répertoire personnel** (`shred` sous Unix, `cipher /W` sous Windows) si l'exfiltration échouait.

Ce qu'il faut en retenir : 1. **LOTL appliqué à la chaîne d'approvisionnement** : l'attaquant utilise un outil légitime, signé, qui ne déclenche aucune alerte antivirus. 2. Le contrôle décisif est d'une simplicité désarmante : `npm ci --ignore-scripts` neutralise les deux vagues. 3. Trois incidents majeurs de 2025 (Nx/s1ngularity, PostHog, AsyncAPI) partagent **un seul anti-patron** : un workflow `pull_request_target` faisant un checkout du code de la pull request.

Sources : Socket <https://socket.dev/blog/tinycolor-supply-chain-attack-affects-40-packages> · Wiz <https://www.wiz.io/blog/shai-hulud-2-0-ongoing-supply-chain-attack> · alerte CISA du 23 septembre 2025

Purger un secret de l'historique

Si, malgré tout, il faut réécrire l'historique (secret encore valide et non révocable, obligation contractuelle) :

OUTIL	STATUT
<code>git filter-repo</code>	L'outil recommandé aujourd'hui. Rapide, sûr, maintenu
BFG Repo-Cleaner	Alternative Java, très rapide sur les gros dépôts

OUTIL

STATUT

`git filter-branch`**Déconseillé par la documentation Git elle-même** : lent et truffé de pièges de sécurité

```
pipx install git-filter-repo

# Supprimer un fichier de tout l'historique
git filter-repo --path secrets/prod.env --invert-paths

# Remplacer un motif partout
printf 'AKIAIOSFODNN7EXAMPLE==>***REVOKED***\n' > motifs.txt
git filter-repo --replace-text motifs.txt
```

▲ LA RÉÉCRITURE D'HISTORIQUE EST UNE OPÉRATION DESTRUCTRICE

Toutes les empreintes de commits changent. Conséquences :

- **tous les collaborateurs doivent recloner** — un `git pull` réintroduirait l'ancien historique ;
- les pull requests ouvertes, les références de tickets et les liens permanents sont cassés ;
- les forks et les copies déjà distribuées conservent le secret ;
- sur GitHub, il faut **contacter le support** pour purger les caches et les vues de commits.

Encore une fois : révoquez d'abord. La réécriture est du nettoyage, pas de la remédiation.

CHAPITRE 5

Signer ses commits

🕒 OBJECTIFS DU CHAPITRE

Comprendre pourquoi l'auteur d'un commit Git n'est pas une preuve, et comment en faire une.

Le problème : l'identité Git n'est pas authentifiée

```
git config user.name "Linus Torvalds"
git config user.email "torvalds@linux-foundation.org"
git commit -m "trust me"
```

Rien ne l'empêche. Les champs auteur d'un commit sont **du texte libre**, saisis par le client. Sur l'interface web, l'avatar affiché est même celui du compte correspondant à l'adresse — ce qui rend l'usurpation visuellement convaincante.

🕒 LE DÉTAIL QUI TRAHIT

Dans l'incident AsyncAPI de juillet 2026, les commits injectés portaient l'identité git **par défaut**, jamais configurée :

```
Author: Your Name <you@example.com>
```

Un indice trivial... que personne n'a vu passer.

Signature GPG ou SSH

Git permet de signer cryptographiquement les commits et les tags. Deux formats coexistent :

FORMAT	AVANTAGE	INCONVÉNIENT
GPG (<code>openpgp</code>)	Historique, standard, révocation	Chaîne d'outils lourde, gestion de clés pénible gérée
SSH (<code>gpg.format = ssh</code>)	Réutilise la clé SSH que vous avez déjà	Vérification hors plateforme nécessite un fichier <code>allowed_signers</code>

La signature SSH est aujourd'hui la voie la plus simple à déployer dans une promotion :

```
# 1. Utiliser la clé SSH existante comme clé de signature
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub

# 2. Signer automatiquement tous les commits et tags
git config --global commit.gpgsign true
```

```
git config --global tag.gpgsign true

# 3. Pour vérifier localement, déclarer les signataires de confiance
echo "$(git config user.email) $(cat ~/.ssh/id_ed25519.pub)" >> ~/.git-allowed-signers
git config --global gpg.ssh.allowedSignersFile ~/.git-allowed-signers

# 4. Vérifier
git log --show-signature -1
git verify-commit HEAD
```

Il faut ensuite **déclarer la clé publique comme clé de signature** sur GitHub ou GitLab (menu SSH keys → type « Signing Key ») pour obtenir le badge *Verified*.

★ CE QUE LA SIGNATURE PROUVE — ET CE QU'ELLE NE PROUVE PAS

Elle prouve que **le détenteur de cette clé privée a produit ce commit**. Elle ne prouve **pas** :

- que le contenu est correct ou sûr ;
- que la personne était autorisée à faire ce changement ;
- que la clé n'a pas été volée.

C'est un maillon d'une chaîne : signature + protection de branche + revue obligatoire + vérifications de CI. Un seul de ces contrôles ne suffit jamais — c'est le principe de **défense en profondeur**.

🔗 EXERCICE 2.2 — PROUVER LA FAIBLESSE, PUIS LA CORRIGER

1. Dans un dépôt jetable, faites un commit en usurpant l'identité de votre formateur (`user.name` / `user.email`). Constatez le résultat de `git log`.
2. Configurez la signature SSH de vos commits.
3. Refaites un commit et vérifiez avec `git log --show-signature`.
4. **Question de réflexion** : la signature empêche-t-elle l'usurpation du champ `Author` ? Que faut-il ajouter côté serveur pour que la garantie soit réelle ?

CHAPITRE 6

TP 1 — Un dépôt IaC sain, de bout en bout

🕒 OBJECTIFS DU CHAPITRE

Mettre en application, sur un dépôt neuf, l'ensemble des garde-fous du module. Ce dépôt servira de base aux TP des modules suivants.

🔧 TP 1 — DURÉE : 1 H 30

■ Partie A — Initialisation (20 min)

- 1 Créez un dépôt `tp-iac-<votrenom>` sur GitHub (ou GitLab), **privé**, sans fichier initial.
- 2 Clonez-le, puis créez à la racine : `.gitattributes`, `.gitignore`, `.editorconfig`, `README.md`, `Makefile`.
- 3 Le `Makefile` doit proposer au minimum `help` (par défaut, auto-documentée), `lint`, `secrets`, `clean`, et déclarer `SHELL := /bin/bash` ainsi que `.SHELLFLAGS := -eu -o pipefail -c`.
- 4 Commitez en respectant *Conventional Commits*. Poussez.

■ Partie B — Garde-fous (30 min)

- 1 Installez `pre-commit` et écrivez un `.pre-commit-config.yaml` contenant au minimum : `trailing-whitespace`, `end-of-file-fixer`, `check-yaml`, `mixed-line-ending (--fix=lf)`, `detect-private-key` et `gitleaks`.
- 2 Lancez `pre-commit install` puis `pre-commit run --all-files`. Corrigez ce qui doit l'être.
- 3 Ajoutez la cible `secrets` au `Makefile` : elle doit lancer `gitleaks detect --source . --verbose`.

■ Partie C — Provoquer la fuite (20 min)

- 1 Créez volontairement un fichier `config/app.env` contenant une **fausse** clé de forme AWS : `text AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY` (Ces valeurs sont les exemples publics de la documentation AWS : elles ne donnent accès à rien.)
- 2 Tentez `git add config/app.env && git commit -m "chore: config"`. **Que se passe-t-il ?** Notez le message exact.
- 3 Contournez le hook avec `--no-verify` et commitez. **Que démontre ce contournement ?**
- 4 Lancez `make secrets`. Le secret est-il détecté ?
- 5 Purgez-le avec `git filter-repo`, puis vérifiez avec `git log --all --full-history -- config/app.env` qu'il a disparu.

■ Partie D — Signature et protection (20 min)

- 1 Configurez la signature SSH de vos commits et déclarez la clé publique sur la plateforme.
- 2 Faites un commit signé, vérifiez le badge *Verified*.
- 3 Activez sur la branche par défaut : interdiction du push direct, une revue obligatoire, interdiction du force push.
- 4 Tentez un `git push` direct sur `main`. **Notez le message de refus.**

■ Livrable

L'URL du dépôt, et un fichier `docs/tp1-rapport.md` répondant à ces quatre questions :

1. Pourquoi `--no-verify` fonctionne-t-il, et quelle est la seule parade réellement efficace ?
2. Le secret que vous avez purgé était-il, à un moment, présent sur le serveur distant ? Qu'auriez-vous dû faire **en premier** s'il avait été réel ?
3. En quoi la mutabilité des tags Git explique-t-elle l'incident `tj-actions/changed-files` ?
4. Citez trois éléments du dépôt qui relèvent de la *gestion de configuration* au sens ITIL du terme.

CE QU'IL FAUT RETENIR DU MODULE 2

- « Gestion de configuration » a **deux sens** : gouvernance (ITIL, CMDB) et outillage (Ansible). En IaC, **le dépôt Git devient la CMDB**.
- La **dérive de configuration** est inévitable ; elle se traite par la **détection automatique** et l'**immutabilité**, pas par la discipline.
- Git stocke des **instantanés adressés par empreinte** : l'historique est infalsifiable, mais **un secret comité y reste**.
- **Un tag est mutable** ; une empreinte de commit ne l'est pas. C'est le cœur de l'incident `tj-actions/changed-files` (mars 2025). Épinglez par SHA.
- **Révoquer d'abord, réécrire l'historique ensuite**. L'inverse donne un faux sentiment de sécurité.
- `pre-commit` est un **filet ergonomique**, pas un contrôle : `--no-verify` le contourne. La barrière est en CI + protection de branche.
- La **signature des commits** transforme un champ de texte libre en preuve cryptographique — mais elle ne prouve ni l'autorisation ni la qualité.
- Trois incidents majeurs (Nx, PostHog, AsyncAPI) partagent un anti-patron unique : `pull_request_target` + **checkout du code de la PR**.